iPhone Security

CS 161

Collaborated on some images with GPT and Gemini

Announcements

- Work with a partner 

Some interesting aspects of iPhone security

Some components (not comprehensive) of securing the phone:



Read more: https://support.apple.com/guide/security/welcome/

Encrypted filesystems

Why encrypt?

- If phone is lost/stolen, don't want others to gain access to data
- Can rapidly erase all data on phone, by just erasing the decryption key

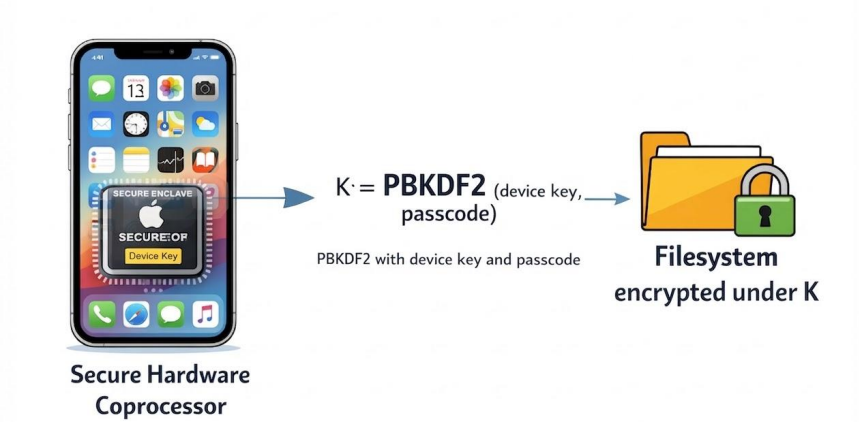
Key management

- Encrypt all data on iPhone, but with what key?
  - Key stored on device? Passcode?



Solution

- Secure hardware coprocessor on phone, used only for security
- Device key hardcoded into coprocessor
- Filesystem encrypted under K, where K = PBKDF2(device key, passcode)
- Coprocessor rate-limits computation of K



Design quiz

- How would you enable someone to authenticate via face authentication instead of passcode?

Encrypted backups

Backups are essential

- If phone is lost/stolen/dropped in a creek, need a way for the owner to buy a new phone and get their data onto the new phone

Design quiz



- How could you design a way to back up the phone to the cloud?
- Requirement: the owner should be able to access their backup, but not other random people

Design quiz #2



- Usability requirement: if phone is lost/stolen, can restore
- Usability requirement: users might forget their device passcode, still need to be able to access their data
- Security requirement: if Apple servers get hacked, hackers shouldn't be able to access users' backup data
  - And if government demands access, Apple should be able to say it has no way to comply
  - Apple shouldn't have access to user's backup data
- Security requirement: user passcodes are often guessable

Solution

- Almost impossible to satisfy all of these at once -- but not totally impossible!
- Apple's solution: Advanced Data Protection
- Opt-in (off by default), backups encrypted under a random key K;
  - save encryption of K under PBKDF2(passcode),
  - encryption of K under a random recovery code (write it down when you enable ADP),
  - encryption of K under a random code given to a contact/family member you designate
- This way, Apple doesn't have the keys and can't decrypt, even if they wanted to (or were hacked, or required by demand from some government somewhere on the planet)
- Similar to Project 2!

End-to-end encryption for iMessage

Design quiz



- iMessage = Apple's text messages
- These messages are encrypted end-to-end
- How could you design such a system? How would the parties obtain or select keys to use to encrypt messages?

Apple's solution

- Each device has a public key
- Similar to Project 2, Apple runs a key server: you can look up a person, and get a list of the public keys of their devices
- Encrypt the message with a random symmetric key K, encrypt K with each public key for each of the recipient's devices, sign the result
  - Enc(K, msg), Enc(K, PK_dst), sign everything
- Send a notification to all of a user's devices, any time a public key is added



- Quiz: What are the security properties of this design? Who can eavesdrop on messages?

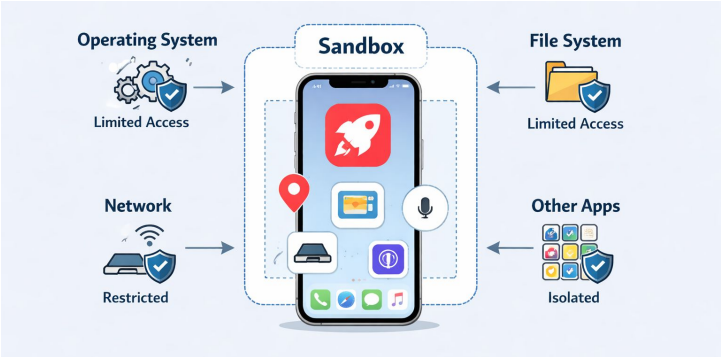
App-oriented security

Multi-user vs multi-app

- Desktop and server OS's are based on a 1970's-era view of computing: computers are really rare, must be shared among users, so the primary threat is another user on the same computer trying to access your data
- Today most devices are used by a single person, and the primary threat is malicious content from the Internet exploiting a vulnerability, or that a third-party application you install is vulnerable or malicious

App sandboxes

- On mobile OS's, each app runs in a sandbox (as a separate "user")
- By default, each app cannot access data stored by another app
- Each app has a separate set of permissions (access to sensors, data, ...)
- Makes it safer to install an app from the app store from someone you've never heard of



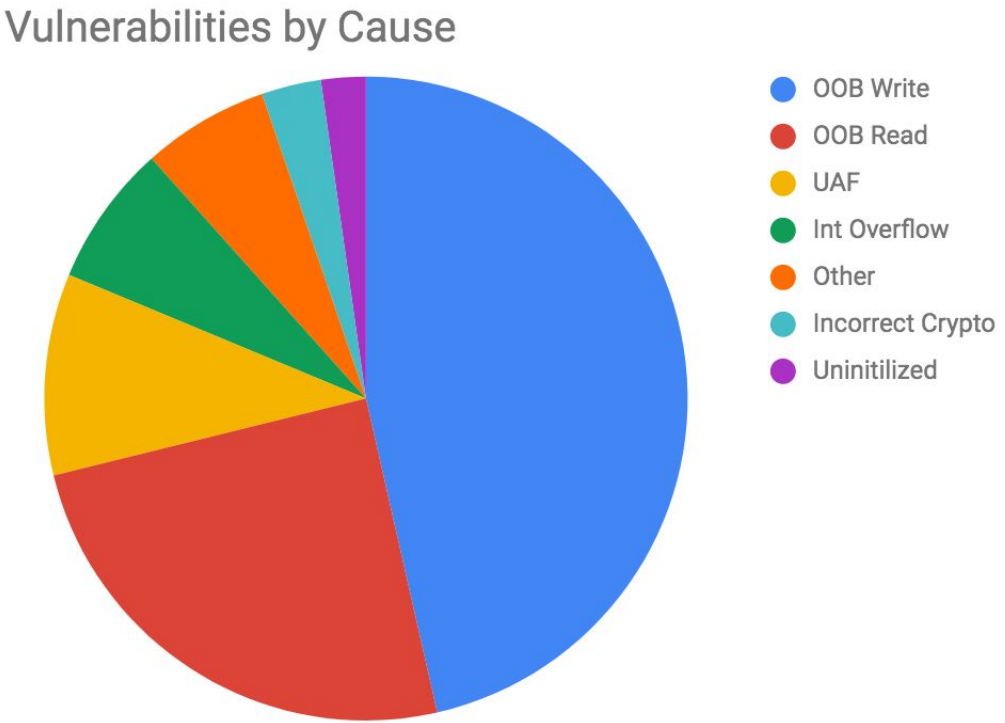
Jailbreaks and software security exploits

Software security

- Software security exploits can be used to hack people's phones
- They can also be used to "jailbreak" your own phone
  - Modify Apple's operating system/apps, e.g., to remove restrictions
- If you find a fully working exploit on iPhone, you can earn $5M
- 2021: NSO Pegasus spyware revealed, used by governments for spying
  - 50,000 phone numbers targeted: journalists, country's opposition politicians, human rights activists, ...
  - Exploits zero-day vulnerabilities in Safari/iMessage/kernel: e.g., out-of-bounds write, information leak, ...

Types of software security vulnerabilities

90% of vulnerabilities are memory safety vulns (Android 2018)



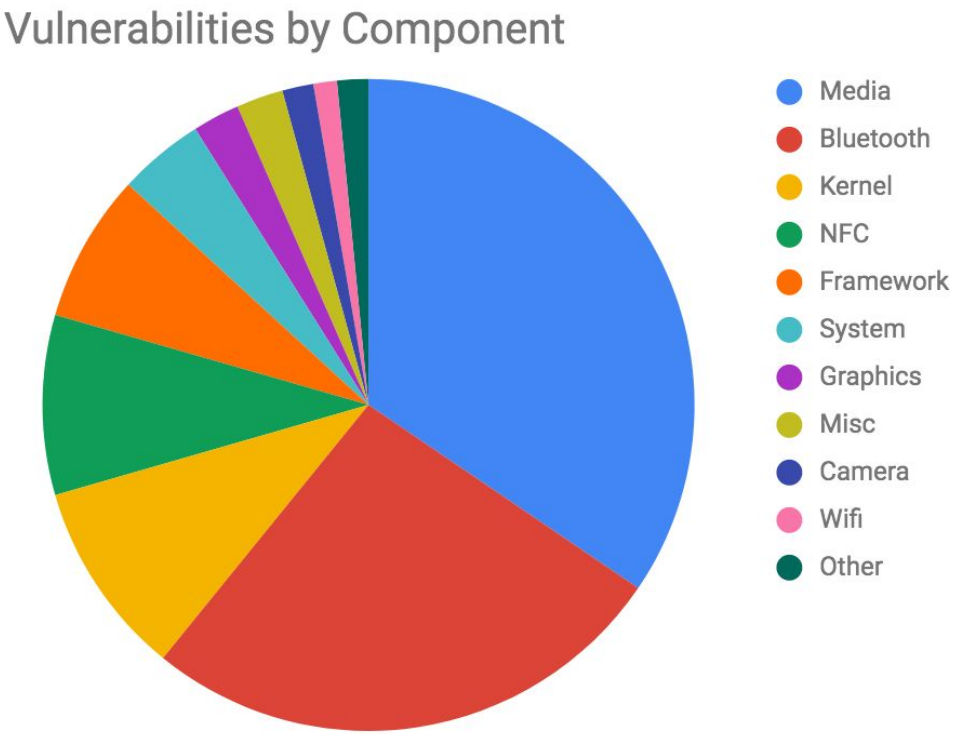
Defenses against memory safety vulnerabilities

- ASLR + DEP
- Pointer Authentication Code (PAC): store a MAC of value of the pointer and the location where the pointer is stored, in the upper bits of the pointer value
  - Extra protection against overwriting function pointers, return addresses, vtables; ROP
  - Hardware-accelerated, for speed
- Memory Tagging Extension (MTE): each memory region has a tag, each pointer has a tag, reading/writing only allowed if both tags match
  - Protection against reading or writing memory out of bounds and use-after-free
  - Each malloc() gets a new tag for the allocated memory region
  - Hardware-accelerated, for speed
- Sandboxing

Sandboxing

More than half of vulns are in software for interacting with data/comms from external world (Android 2018)

⇒ Sandbox media codecs, iMessage attachments, HTML/CSS/JS in web browser, ...



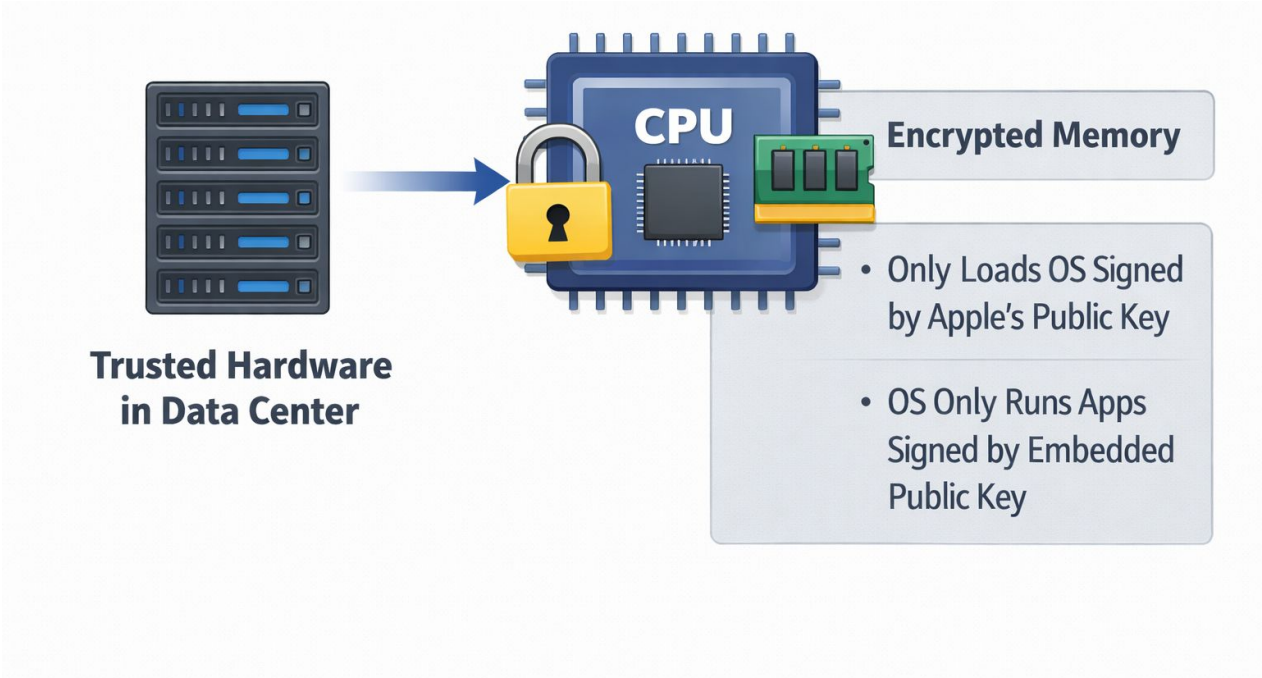
Private cloud compute

Motivation

- Some compute tasks are too heavy to perform on the phone -- e.g., speech-to-text, AI inference
- Solution: run on Apple's cloud
- Goal: make this as secure as local computation
  - Even Apple, and Apple's employees, should not be able to gain access to this data
  - Even if Apple is hacked, hackers can't gain access to this data
  - Even if some government somewhere demands access, Apple can't provide it

Trusted hardware/TEE/enclave

- Trusted hardware in cloud data centers
- CPU in trusted hardware encrypts all data stored in memory



Design

- Hardware will only load OS that is signed by a hardcoded public key pair of Apple
- OS will only load and run applications that are signed and authorized by a public key pair that is embedded in the OS by Apple
- Apple releases source code, binaries for all authorized OS versions and applications
- OS/applications written to be stateless (delete all data after finishing a computation) and won't reveal data to anyone else
- No one, not even Apple, can access the data in trusted hardware

Attestation

- Trusted hardware has a unique device public/private key pair
- Trusted hardware keeps track of what OS was loaded, and will send a signed statement about the OS that was loaded (signed with its private key) to the client
- OS keeps track of what applications it loaded, and will send a signed statement about what applications were loaded
- Allows client to verify that it is running proper software -- for instance, a hacker who breaks into Apple's cloud can't modify the OS or application without being detected

Transparency log

- Apple maintains a public transparency log of all OS's/applications that are authorized, and trusted hardware verifies that OS/application is present in the log before loading it
- Uses chaining: Sign(Sign(Sign(Sign(a1) || a2) || a3) || a4),
  where a1, a2, a3, a4 are the authorized applications
- Log is append-only: can append a5, to get
  Sign(Sign(Sign(Sign(Sign(a1) || a2) || a3) || a4) || a5);
  trusted hardware will only accept a new chain if it has the form
  Sign(old_chain || something)
- Q: what happens if the log tries to fork a chain, e.g. signing a6, on top of a4?

Summary

- Phones are an interesting applied example of the latest, greatest technology in computer security
- Multi-layered security includes:
  - Encrypted filesystems
  - Encrypted backups
  - Encrypted messaging
  - App-oriented security
  - Private cloud